How to run

•Connect with SQL*Plus and run the file as a script: sqlplus user/pass@db, then @sqlid_diagnostic_v2_4.sql (do not paste it line by line).

•When prompted, enter a SQL_ID or a SID. A SID is resolved to its SQL_ID automatically.

•The report is written as an HTML file named sqlid_diag_<sql_id>_<timestamp>.html in the current directory.

Below is a short summary of each section, how to read it, and a small screenshot for both cases: when the SQL_ID is running and when it is not running. Every value in them is invented sample data, not from a real database.

Quick reading order

- **1.1** first: it tells you RUNNING or NOT RUNNING, which decides what the other sections show.
- **1.4 and 3** next: is this run slower than before, and did the plan hash value change?
- **4** then: is the time spent waiting, and is somebody blocking?
- **5.3 and 6.2** last: which plan is slower, and what do the findings recommend?

At a glance: what you see in each case

Section	SQL_ID running	SQL_ID not running
1.1 Execution status	RUNNING; session, start time, plan and child cursor followed	NOT RUNNING
1.3 Sessions	Live session list	Skipped (note shown)
1.4 SQL Monitor summary	Current execution plus recent ones	Last 10 executions
1.5 SQL Monitor report	Live report, plan, outline, predicates, binds	Skipped (note shown)
1.6 Cursor cache plan	Not repeated (plan is shown in 1.5)	Plan, outline, binds of latest child cursor
1.7 Best AWR plan	Best historical plan	Best historical plan
Plan RCA / RCA.2	Shown with live figures	Not displayed
2, 3 History and signature	Shown	Shown
4.1 Current waits	Live waits	No current waits
4.2 Blocking chain	Live blockers	Blocking of the last execution from ASH
4.3 Wait guidance	Cause and resolution per wait event	No active waits
5.3 Plan history	Shown	Shown
6.2 Findings	Top High-priority findings	Top High-priority findings

Section-by-section guide

1 Current Execution Status, SQL Monitor and Execution Plan: 1.1 Execution status

What it shows: Whether any session is executing the SQL_ID right now (active sessions and SQL Monitor entries in EXECUTING state), and which execution the report follows.

How to read it: Read this first. A RUNNING badge fills the live sections; NOT RUNNING makes them show a skip note and points you to Sections 3 and 5 for earlier executions.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

1.1 Execution status

Result	RUNNING / IN PROGRESS at least one session is executing this SQL_ID right now.
Active sessions executing this SQL_ID	1
SQL Monitor executions in state EXECUTING	1
Execution followed in this report (most recently started)	SID **1847** (serial# 22531), started 2026-10-22 07:58:41, plan hash value 3284719650, child cursor 4

Running: 1 active session; report follows SID 1847, started 07:58:41, plan hash 3284719650, child cursor 4.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

1.1 Execution status

Result	NOT RUNNING no session is executing this SQL_ID (no ACTIVE session and no SQL Monitor entry in state EXECUTING).
Active sessions executing this SQL_ID	0
SQL Monitor executions in state EXECUTING	0

Not running: 0 active sessions and 0 EXECUTING entries, so live sections are skipped.

1.3 Sessions currently running this SQL_ID

What it shows: The sessions executing the statement: SID, serial#, user, status, time in the current state, wait class and current or last wait event.

How to read it: Check who runs it, for how long, and what it is waiting on. A long time in one state with a User I/O or lock wait tells you where to look next.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

1.3 Sessions currently running this SQL_ID

SID	Serial#	User	Session Status	This SQL_ID is	SQL Exec Start	In Current State For	Wait Class	Current / Last Wait Event
1847	22531	APP_REPORTING	ACTIVE	Current statement	2026-10-22 07:58:41	1 hr 53 min	User I/O	db file sequential read

Running: SID 1847 (APP_REPORTING) active for 1 hr 53 min, waiting on db file sequential read (User I/O).

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

1.3 Sessions currently running this SQL_ID

> Session details is skipped: Section 1.1 reports this SQL_ID as **NOT RUNNING**, so there is no current execution to show. Information about earlier executions is in Sections 3 and 5.

Not running: skipped, with a pointer to Sections 3 and 5.

1.4 Real-time SQL Monitor summary (10 most recent executions)

What it shows: Elapsed, CPU, buffer gets, physical reads, rows processed, plan hash value and work-area size for the latest monitored executions.

How to read it: Compare the latest row with earlier ones. Elapsed time up with a new plan hash value points to a plan change; elapsed time up with the same plan hash value points to waits, locks or data volume rather than the plan.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

1.4 Real-time SQL Monitor summary (10 most recent executions)

SID	SQL_ID	SQL Exec ID	Status	Monitoring Start	SQL Exec Start	Elapsed (s)	CPU (s)	Buffer Gets	Physical Reads (blocks)	Rows Processed	Plan Has
1847	7h3k9fqz2m8xt	16777230	EXECUTING	2026-10-22 07:58:46	2026-10-22 07:58:41	6,812.40	2,104.77	48,331,907	6,902,114	-	3284
1612	7h3k9fqz2m8xt	16777199	DONE (ALL ROWS)	2026-10-19 02:00:03	2026-10-19 02:00:03	412.35	188.20	3,904,552	511,980	6	1952
1590	7h3k9fqz2m8xt	16777150	DONE (ALL ROWS)	2026-10-18 02:00:04	2026-10-18 02:00:04	398.90	181.05	3,870,118	498,327	6	1952

Running: the EXECUTING row is at 6,812 s on plan 3284719650, against about 400 s earlier on plan 1952360471.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

1.4 Real-time SQL Monitor summary (10 most recent executions)

SID	SQL_ID	SQL Exec ID	Status	Monitoring Start	SQL Exec Start	Elapsed (s)	CPU (s)	Buffer Gets	Physical Reads (blocks)	Rows Processed	Plan Has
2210	4c8vz1nq6d0yp	16777388	DONE	2026-10-22 03:14:25	2026-10-22 03:14:20	5,430.12	1,902.40	38,114,220	1,402,118	1,204,330	2718
2174	4c8vz1nq6d0yp	16777301	DONE	2026-10-21 03:14:23	2026-10-21 03:14:18	238.44	181.10	3,402,873	131,902	1,188,917	2718
2161	4c8vz1nq6d0yp	16777219	DONE	2026-10-20 03:14:22	2026-10-20 03:14:17	211.80	176.52	3,377,640	129,311	1,171,206	2718

Not running: latest run took 5,430 s against 238 s and 212 s before, all on the same plan 2718459031, so the plan is not the cause here.

1.5 SQL Monitor report (statement is executing now)

What it shows: The live SQL Monitor text report: global information, global stats (elapsed, CPU, I/O) and a step-by-step plan with estimated rows, actual rows, executions and activity %. Below it come the plan, outline hints, predicate information and the bind values in use.

How to read it: Find the step with the highest Activity (%). Compare Rows (Estim) with Rows (Actual) and look at Execs: a big gap or a huge execution count on a nested-loops inner side is a classic bad-estimate signature.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

1.5 SQL Monitor report (statement is executing now)

Executing statement followed here: SID **1847**, started 2026-10-22 07:58:41, SQL_EXEC_ID 16777230, child cursor **4**, plan hash value **3284719650**.

▼ Show SQL Monitor report

```
SQL Monitoring Report

SQL Text
------------------------------
SELECT c.region, o.order_status, COUNT(DISTINCT o.order_id) AS orders, ...

Global Information
------------------------------
 Status              :  EXECUTING
 Instance ID         :  1
 Session             :  APP_REPORTING (1847:22531)
 SQL ID              :  7h3k9fqz2m8xt
 SQL Execution ID    :  16777230
 Execution Started   :  10/22/2026 07:58:41
 First Refresh Time  :  10/22/2026 07:58:46
 Last Refresh Time   :  10/22/2026 09:52:13
 Duration            :  6812s
 Module/Action       :  ReportBatch/DAILY_REVENUE
 Service             :  SVC_REPORTING
```

Running (top of the report shown): step 10 carries 64.2% of activity with 3.1M executions; step 7 estimated 1,200 rows against 612K actual.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

1.5 SQL Monitor report (statement is executing now)

> SQL Monitor report is skipped: Section 1.1 reports this SQL_ID as **NOT RUNNING**, so there is no current execution to show. Information about earlier executions is in Sections 3 and 5.

Not running: skipped, because there is no current execution to monitor.

1.6 Execution plan from the cursor cache

What it shows: The plan of the most recently active child cursor with outline, predicate information, peeked binds and captured binds.

How to read it: Look at access methods (full scan or index), join methods, estimated rows (E-Rows) and the predicates marked with * to see how the filters are applied. Check the bind values the plan was built with.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

1.6 Execution plan from the cursor cache

> The current execution plan, outline, predicate information and bind values of the executing statement are shown in Section 1.5, so they are not repeated here.

Running: not repeated, because the executing statement's plan is already shown under 1.5.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

1.6 Execution plan from the cursor cache

Selected cursor: child number **0**, plan hash value **2718459031** (the most recently active of 1 child cursor(s) that have a plan). Plan, outline and bind values below all come from this one child cursor.

▼ Show plan

```
SQL_ID  4c8vz1nq6d0yp, child number 0
-------------------------------------
Plan hash value: 2718459031

-----------------------------------------------------------------------------------------------
| Id  | Operation                          | Name               | E-Rows | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | UPDATE STATEMENT                   |                    |        | 18420  (2) |          |
|   1 |  UPDATE                            | INVENTORY_SNAPSHOT |        |            |          |
|*  2 |   HASH JOIN RIGHT SEMI             |                    |  1175K | 18420  (2) | 00:03:41 |
|*  3 |    TABLE ACCESS FULL               | STOCK_MOVEMENTS    |   902K |  9611  (2) | 00:01:56 |
|   4 |    TABLE ACCESS BY INDEX ROWID BATCHED| INVENTORY_SNAPSHOT |  1190K |  8702  (1) | 00:01:45 |
|*  5 |     INDEX RANGE SCAN               | INV_WH_IX          |  1190K |   611  (1) | 00:00:08 |
-----------------------------------------------------------------------------------------------

Peeked Binds (identified by position):
--------------------------------------
   1 - :B1 (NUMBER): 14
   2 - :B2 (DATE): 10/21/2026 00:00:00
```

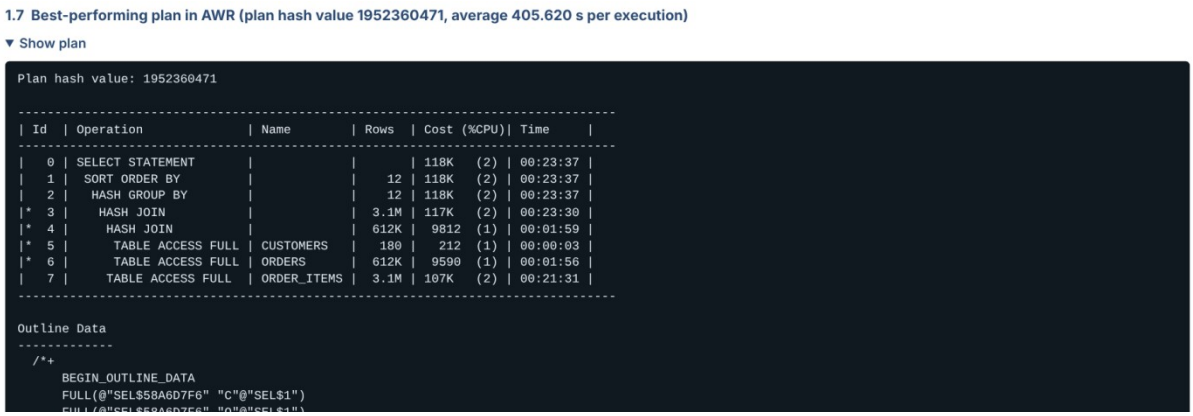
Not running (top shown): child 0, plan 2718459031, a hash join with a full scan of STOCK_MOVEMENTS and an index range scan on INV_WH_IX.

1.7 Best-performing plan in AWR

What it shows: The plan with the lowest average elapsed time in AWR, with outline, predicates and the bind values recorded in the snapshot.

How to read it: Use it as the "what good looked like" reference. Compare its operations and binds with the current plan to see what changed. When it is the same plan as the current one, a note says so.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)



1.7 Best-performing plan in AWR (plan hash value 1952360471, average 405.620 s per execution)

▼ Show plan

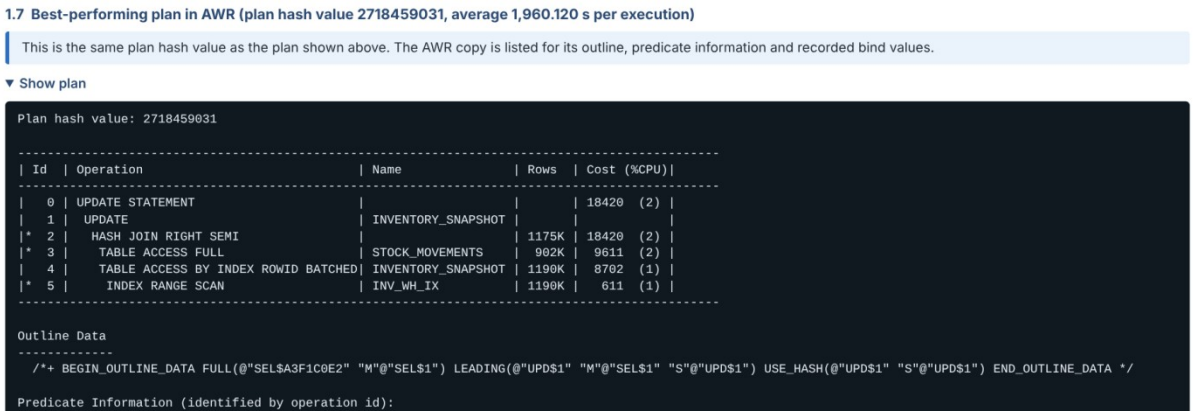
```
Plan hash value: 1952360471

-------------------------------------------------------------------------------
| Id  | Operation              | Name        | Rows  | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |             |       | 118K   (2) | 00:23:37 |
|   1 |  SORT ORDER BY         |             |    12 | 118K   (2) | 00:23:37 |
|   2 |   HASH GROUP BY        |             |    12 | 118K   (2) | 00:23:37 |
|*  3 |    HASH JOIN           |             |  3.1M | 117K   (2) | 00:23:30 |
|*  4 |     HASH JOIN          |             |  612K |  9812  (1) | 00:01:59 |
|*  5 |      TABLE ACCESS FULL | CUSTOMERS   |   180 |   212  (1) | 00:00:03 |
|*  6 |      TABLE ACCESS FULL | ORDERS      |  612K |  9590  (1) | 00:01:56 |
|   7 |     TABLE ACCESS FULL  | ORDER_ITEMS |  3.1M | 107K   (2) | 00:21:31 |
-------------------------------------------------------------------------------

Outline Data
-------------
  /*+
      BEGIN_OUTLINE_DATA
      FULL(@"SEL$58A6D7F6" "C"@"SEL$1")
```

Running (top shown): best plan 1952360471, 405.6 s on average, built on hash joins and full scans; the current plan uses nested loops.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)



1.7 Best-performing plan in AWR (plan hash value 2718459031, average 1,960.120 s per execution)

This is the same plan hash value as the plan shown above. The AWR copy is listed for its outline, predicate information and recorded bind values.

▼ Show plan

```
Plan hash value: 2718459031

-----------------------------------------------------------------------------------------
| Id  | Operation                          | Name               | Rows  | Cost (%CPU)|
-----------------------------------------------------------------------------------------
|   0 | UPDATE STATEMENT                   |                    |       | 18420  (2) |
|   1 |  UPDATE                            | INVENTORY_SNAPSHOT |       |            |
|*  2 |   HASH JOIN RIGHT SEMI             |                    | 1175K | 18420  (2) |
|*  3 |    TABLE ACCESS FULL               | STOCK_MOVEMENTS    |  902K |  9611  (2) |
|   4 |    TABLE ACCESS BY INDEX ROWID BATCHED| INVENTORY_SNAPSHOT | 1190K |  8702  (1) |
|*  5 |     INDEX RANGE SCAN               | INV_WH_IX          | 1190K |   611  (1) |
-----------------------------------------------------------------------------------------

Outline Data
-------------
  /*+ BEGIN_OUTLINE_DATA FULL(@"SEL$A3F1C0E2" "M"@"SEL$1") LEADING(@"UPD$1" "M"@"SEL$1" "S"@"UPD$1") USE_HASH(@"UPD$1" "S"@"UPD$1") END_OUTLINE_DATA */

Predicate Information (identified by operation id):
```

Not running: the best AWR plan is the same plan 2718459031 as in 1.6, listed for its outline, predicates and AWR binds.

Plan RCA (Current Plan): RCA.2 Summary

What it shows: Shown only while the SQL_ID is running. It analyses the running plan using live figures and gives findings by severity, top concerns, "What to do next" and "What NOT to do".

How to read it: Start with Findings by severity and Top performance concerns. Under "What to do next" each item says what was seen, what to do and how to confirm it worked. Read "What NOT to do" before touching production (for example do not kill the session just because it is slow, do not change several things at once).

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

RCA.2 Summary

SQL_ID and execution status	**7h3k9fqz2m8xt** RUNNING
Child cursor / plan hash value	4 / 3284719650
Plan operations analysed	11
Findings by severity	HIGH 2 | MEDIUM 1 | LOW 0 | INFORMATIONAL 2
Top performance concerns	1. HIGH Running execution already exceeds the estimate by at least 510x at step 7 2. HIGH Step 10 executed 3,148,220 times (nested-loops inner side) 3. MEDIUM HASH GROUP BY at step 2 uses temp space
What to do next	1. **Running execution already exceeds the estimate by at least 510x at step 7** What was seen: Confirmed (lower bound): the live row count is still growing, so the real error is at least this large. The cause is not yet known. **What to do:** Check the statistics and predicates of the objects below this step (see the plan) before changing anything. ORDERS and ORDER_ITEMS statistics should be reviewed first (see Section 5.4: ORDER_ITEMS is flagged STALE). **How to confirm it worked:** Look at the same step again in SQL Monitor (Section 1.5) or re-run this report; the live row count only grows. 2. **Step 10 executed 3,148,220 times (nested-loops inner side)** What was seen: Confirmed: the outer input has already delivered at least 3,148,220 rows against an estimate of 1,450. The join method was chosen for far fewer rows.

Running (top shown): 2 High and 1 Medium finding, such as a 510x estimate miss at step 7 and step 10 executed 3,148,220 times.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

RCA Plan RCA (Current Plan)

Plan RCA (Current Plan): Not displayed because the SQL_ID is not currently running or the query has completed.

Not running: the section is displayed with a note that it is not shown because the statement is not currently running.

2 New SQL_ID Check and Force Matching Signature

What it shows: Whether the SQL_ID has been seen before (child cursors in V$SQL, rows in AWR) and its FORCE_MATCHING_SIGNATURE, with other SQL_IDs that share it.

How to read it: NEW means there is no history to compare against. NOT NEW means history exists. SQL_IDs sharing a signature are the same statement with different literals, so one of them may show a better plan.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

2 New SQL_ID Check and Force Matching Signature

2.1 History check

Result	NOT NEW the SQL_ID has been seen before.
Child cursors in cursor cache (V$SQL)	2
Rows in AWR history (DBA_HIST_SQLSTAT)	41
FORCE_MATCHING_SIGNATURE	11873420956127408331

2.2 SQL_IDs sharing the same FORCE_MATCHING_SIGNATURE

SQL_ID	Force Matching Signature	Distinct Plans	Executions	First Load Time	Last Active Time
7h3k9fqz2m8xt analysed	11873420956127408331	2	42	2026-09-10 02:00:02	2026-10-22 09:52:11

Running: NOT NEW, 2 child cursors, 41 AWR rows, 2 distinct plans for this signature.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

2 New SQL_ID Check and Force Matching Signature

2.1 History check

Result	NOT NEW the SQL_ID has been seen before.
Child cursors in cursor cache (V$SQL)	1
Rows in AWR history (DBA_HIST_SQLSTAT)	67
FORCE_MATCHING_SIGNATURE	6402981175529340118

2.2 SQL_IDs sharing the same FORCE_MATCHING_SIGNATURE

SQL_ID	Force Matching Signature	Distinct Plans	Executions	First Load Time	Last Active Time
4c8vz1nq6d0yp analysed	6402981175529340118	1	3	2026-10-20 03:14:17	2026-10-22 04:44:50

Not running: NOT NEW, 1 child cursor, 67 AWR rows, 1 plan.

3 Execution History: Plan Hash, Average Elapsed, Average Logical I/O

What it shows: AWR history grouped by plan hash value: snapshots, executions, average elapsed time, buffer gets and disk reads, first and last seen.

How to read it: One row means one plan. Two or more rows mean plan flips: compare average elapsed and buffer gets, and use First Seen to link the change to an event such as a statistics gather or deployment.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

3 Execution History - Plan Hash, Average Elapsed, Average Logical I/O

3.1 AWR history grouped by plan (shows plan flips, if any)

Plan Hash Value	Snapshots	Executions	Avg Elapsed (s)	Avg Buffer Gets	Avg Disk Reads	First Seen	Last Seen
3284719650	3	3	2,566.810	18,420,775	2,611,903	2026-10-20 01:00	2026-10-22 03:00
1952360471	38	38	405.620	3,880,114	503,742	2026-09-10 01:00	2026-10-19 03:00

Running: plan 3284719650 (first seen 2026-10-20) averages 2,566.8 s against 405.6 s for the earlier plan 1952360471.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

3 Execution History - Plan Hash, Average Elapsed, Average Logical I/O

3.1 AWR history grouped by plan (shows plan flips, if any)

Plan Hash Value	Snapshots	Executions	Avg Elapsed (s)	Avg Buffer Gets	Avg Disk Reads	First Seen	Last Seen
2718459031	67	67	312.480	4,611,904	168,330	2026-08-17 03:00	2026-10-22 05:00

Not running: a single plan 2718459031 for the whole history, averaging 312.5 s.

4.1 Current waits for sessions running this SQL_ID

What it shows: Wait class, wait event, wait duration, whether the session is blocked, the blocking SID and the decoded wait parameters.

How to read it: If Blocked is Yes, go to 4.2. If not, the wait event shows where the time goes (for example single-block I/O, locks or CPU).

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

4.1 Current waits for sessions running this SQL_ID

1 session(s) running this SQL_ID: 1 currently waiting, none blocked by another session.

SID	Serial#	User	Session Status	State	Wait Class	Wait Event	Wait Duration	Blocked?	Blocking SID	Blocking Status	Wait Parameters (decoded)
1847	22531	APP_REPORTING	ACTIVE	WAITING	User I/O	db file sequential read	0 sec	No	-	NO HOLDER nothing is blocking	file#=27, block#=1048211, blocks=1

Wait duration and wait event are shown only for sessions in state WAITING; any other session is working. Blocking SID is filled only while a session is blocked.

Running: SID 1847 waits on db file sequential read (file#=27, block#=1048211), not blocked.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

4.1 Current waits for sessions running this SQL_ID

No sessions are currently executing this SQL_ID, so there are no current waits to report.

Not running: no sessions are executing the SQL_ID, so there are no current waits.

4.2 Blocking chain: who is blocking, and what are they doing

What it shows: The blocked session, the blocking session (SID, user, state, last SQL_ID) and what the blocker is doing. If the SQL_ID is not running, it shows the blocking recorded for its last execution from ASH.

How to read it: Check Approx. Blocked Time and the blocker status. An idle blocker that holds a lock usually has an open, uncommitted transaction.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

4.2 Blocking chain: who is blocking, and what are they doing

No blocking session detected.

Running: no blocking session detected.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

4.2 Blocking chain: who is blocking, and what are they doing

This SQL_ID is not running (Section 1.1). Blocking recorded for its **last execution** (SQL_EXEC_ID 16777388, started 2026-10-22 03:14:20) from V$ACTIVE_SESSION_HISTORY:

BLOCKED SESSION (this SQL_ID)						→ BLOCKING SESSION			
Blocked SID	Serial#	Blocked User	Waiting On	Approx. Blocked Time	First / Last Sample	Blocking SID	Serial#	Blocking User	Blocking Session No
2210	40117	APP_BATCH	enq: TX - row lock contention	38 min 10 sec	2026-10-22 03:16:40 / 2026-10-22 03:54:50	**874**	9021	FINANCE_APP	INACTIVE (idle) idle for 5 hr 55 min

ASH samples active sessions about once per second (AWR keeps one sample in ten), so blocked time is approximate. Blocker details come from the same ASH samples when the blocker was active, and from V$SESSION (matched on SID and serial#) when it is still connected.

Not running: SID 2210 waited about 38 min 10 s on enq: TX - row lock contention, blocked by SID 874 (FINANCE_APP), idle for 5 hr 55 min.

4.3 Wait-class resolution guidance (for waits happening right now)

What it shows: For each current wait event: wait class, probable cause and a suggested resolution.

How to read it: Use it as a starting hint for the wait you see now, then confirm with the evidence in the other sections.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

4.3 Wait-class resolution guidance (for waits happening right now)

Wait Event	Wait Class	Probable Cause	Suggested Resolution
db file sequential read	User I/O	Single-block I/O, typically index lookups.	Check index selectivity and storage/IO latency; verify AWR IO latency; consider caching hot blocks.

Running: db file sequential read (User I/O): single-block I/O, typically index lookups; check index selectivity and I/O latency.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

4.3 Wait-class resolution guidance (for waits happening right now)

No active non-idle wait events found for this SQL_ID right now.

Not running: no active non-idle wait events to report.

5 Plan Stability Check: 5.3 Plan history and execution performance by plan

What it shows: A verdict on plan stability (5.1), key facts (5.2), and per plan: executions in AWR and cache, average elapsed and "Relative to Fastest Plan" (5.3). Section 5 continues with table and index statistics, fragmentation, last DDL/DML and optimizer parameter changes in the last 7 days.

How to read it: 1.0x is the fastest plan; 6.3x means an execution takes about 6.3 times as long as with the fastest plan. Then check the statistics status and DML volume below it, because stale statistics are a common reason for a plan change.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

5.3 Plan history and execution performance by plan

Plan Hash Value	Observed In	Executions (AWR)	Executions (cache)	Avg Elapsed (s)	Relative to Fastest Plan	First Observed (AWR)	Last Observed
3284719650 current	Cache and AWR	3	1	2,566.810	6.3x	2026-10-20 01:00	2026-10-22 09:52
1952360471	Cache and AWR	38	38	405.620	1.0x	2026-09-10 01:00	2026-10-19 03:00

Relative to Fastest Plan = average elapsed time per execution of the plan divided by that of the fastest plan (plan 1952360471, 405.620 s). **1.0x** means this is the fastest plan; **2.0x** means an execution takes twice as long as with the fastest plan. Here plan 3284719650 shows **6.3x**: on average it takes about 6.3 times as long per execution as plan 1952360471. The value compares plans of the same SQL_ID only; it does not say how fast the fastest plan is in absolute terms.

Running: current plan 3284719650 is 6.3x slower than plan 1952360471.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

5.3 Plan history and execution performance by plan

Plan Hash Value	Observed In	Executions (AWR)	Executions (cache)	Avg Elapsed (s)	Relative to Fastest Plan	First Observed (AWR)	Last Observed
2718459031 current	Cache and AWR	67	3	312.480	1.0x	2026-08-17 03:00	2026-10-22 05:00

Relative to Fastest Plan = average elapsed time per execution of the plan divided by that of the fastest plan (plan 2718459031, 312.480 s). **1.0x** means this is the fastest plan. The value compares plans of the same SQL_ID only; it does not say how fast the fastest plan is in absolute terms.

Not running: one plan at 1.0x (STABLE PLAN).

6 Findings and Recommendations: 6.2 Findings and recommendations

What it shows: First the measured workload (6.1: CPU, I/O, lock and other wait shares, buffer gets, physical reads), then the most important High-priority findings, each with priority, evidence from this SQL_ID and a recommended action.

How to read it: Read the Main finding banner, then the table. Recommended statements are printed as text only and are never run by the script. Test them first and compare plan hash value, elapsed time and buffer gets before and after.

A. SQL_ID is RUNNING (sample SQL_ID 7h3k9fqz2m8xt)

6.2 Findings and recommendations

Main finding: Stale optimizer statistics on SALES.ORDER_ITEMS. 3 High-priority performance finding(s) identified; the 3 most important are listed below.

#	Priority	Finding	Evidence from this SQL_ID	Recommended action
1	High	**Stale optimizer statistics on SALES.ORDER_ITEMS**	Oracle flags the statistics as stale (STALE_STATS = YES). They were gathered 2026-10-12 22:30:02 when NUM_ROWS was 87,155,247; 9,244,871 inserts, 118,774 updates and 0 deletes are recorded since the statistics were gathered (10.8% of NUM_ROWS; counters last flushed 2026-10-22 09:40:02). The row estimate on this table is 510.0x off (estimated 1,200, actual 612,000 per start).	EXEC DBMS_STATS.GATHER_TABLE_STATS(ownname ⇒ 'SALES', tabname ⇒ 'ORDER_ITEMS', cascade ⇒ TRUE); Export the current statistics first (DBMS_STATS.EXPORT_TABLE_STATS) so the change can be reverted, and run it off-peak: dependent cursors are invalidated and may be re-optimised.
2	High	**Elapsed time is dominated by user I/O (68%)**	1 hr 17 min of the 1 hr 53 min total elapsed time (68.0%) is spent waiting for user I/O. Per execution: 6,902,114 physical reads and 48,331,907 buffer gets, so 14.3% of the block requests go to disk; 6,812.400 s per execution over 1 execution(s).	The plan has no full scan of the large tables, so the physical reads come from index and table block access. Look at the index findings (selectivity, clustering factor) and at the hot plan step (step 10: 3.1 million single-row lookups into ORDER_ITEMS).
3	High	**Plan instability: plan 3284719650 is 6.3x slower than plan 1952360471**	2 plan hash values have been observed for this SQL_ID and 2 child cursors exist (1 distinct optimizer environment(s)). Plan 1952360471 is the fastest at 405.620 s per execution, plan 3284719650 the slowest at 2,566.810 s (6.3x). The analysed plan is 3284719650, the slowest one. The analysed plan was first seen in AWR at 2026-10-20 01:00. Cursor flags: 0 child(ren) use a SQL plan baseline, 0 a SQL profile, 0 a SQL patch; 1 bind-sensitive, 0 bind-aware, 1 marked for re-optimisation.	First find out why the plan changed (the stale statistics finding above is the prime candidate). To keep the fast plan meanwhile, load it as a baseline while it is still in the cursor cache: DECLARE n PLS_INTEGER; BEGIN n := DBMS_SPM.LOAD_PLANS_FROM_CURSOR_CACHE(sql_id ⇒ '7h3k9fqz2m8xt', plan_hash_value ⇒ 1952360471); END; (SQL Plan Management needs Enterprise Edition.) Some cursors are bind-sensitive, so compare the peeked binds in Section 1.5 before pinning a single plan.

Running: stale statistics on SALES.ORDER_ITEMS, user I/O at 68% of elapsed, and a plan 6.3x slower than the best plan.

B. SQL_ID is NOT RUNNING (sample SQL_ID 4c8vz1nq6d0yp)

6.2 Findings and recommendations

Main finding: Time lost to row-lock (application) waits: 39%. 1 High-priority performance finding(s) identified; the 1 most important are listed below.

#	Priority	Finding	Evidence from this SQL_ID	Recommended action
1	High	**Time lost to row-lock (application) waits: 39%**	38 min 30 sec of 1 hr 38 min elapsed (39.3%) is application wait, which is mostly row-lock and enqueue waiting on other sessions' transactions. ASH shows SID 2210 waiting on `enq: TX - row lock contention` for about 38 min 10 sec during the last execution (03:16:40 to 03:54:50), blocked by SID 874 (FINANCE_APP), which was not active while it blocked and is idle now.	The statement is slow because it waits for locks held by other transactions, not because of its plan. Identify the blocker (SID 874, FINANCE_APP), shorten the blocking transaction (commit sooner, no think time while rows are locked), and check for unindexed foreign keys on the tables it modifies.

The plan itself shows no High-priority issue: the plan is stable, statistics are not stale, and the two earlier runs took about 4 minutes each.

Not running: 39% of elapsed time lost to row-lock waits caused by SID 874; the plan itself is fine.

Oracle tables and views used to populate the report

The list below was compiled from the script source. Everything is read with SELECT statements; the script starts with SET TRANSACTION READ ONLY, and DBMS_STATS, DBMS_SPM and similar commands that appear in recommendations are only printed as text, never executed. Grant SELECT directly (not through a role) to the reporting account. AWR, ASH and SQL Monitor .

Oracle view / package	Used for	Main sections
V$DATABASE, V$INSTANCE	Database name, instance, version, role	Header
V$SESSION (GV$SESSION), V$PX_SESSION	Active sessions, current waits, blockers, parallel sessions	1.1, 1.3, 4.1, 4.2
V$SQL_MONITOR, V$SQL_PLAN_MONITOR	Recent monitored executions, live row counts, work areas	1.1, 1.4, 1.5, RCA
DBMS_SQLTUNE (REPORT_SQL_MONITOR)	SQL Monitor text report	1.5
VSQL, VSQLAREA, V$SQLSTATS	SQL text, child cursors, load times, workload figures	1.2, 2, 5, 6.1
VSQL_PLAN, VSQL_PLAN_STATISTICS_ALL, DBMS_XPLAN	Plans from the cursor cache with outline and predicates	1.5, 1.6, RCA
V$SQL_BIND_CAPTURE	Captured bind values	1.5, 1.6
V$SQL_WORKAREA, V$SQL_WORKAREA_ACTIVE, V$TEMPSEG_USAGE	Memory, temp usage and spills of sorts and hash joins	RCA, 6
VSQL_SHARED_CURSOR, VPARAMETER	Why child cursors differ, optimizer settings	5, 6

Oracle view / package	Used for	Main sections
V$ACTIVE_SESSION_HISTORY, DBA_HIST_ACTIVE_SESS_HISTORY	Waits and blocking of past executions	4.2, 6
DBA_HIST_SQLSTAT, DBA_HIST_SNAPSHOT	Execution history by plan	1.7, 2, 3, 5.3
DBA_HIST_SQL_PLAN, DBA_HIST_SQLTEXT, DBA_HIST_SQLBIND	AWR plans, SQL text, bind values	1.2, 1.7
DBA_HIST_PARAMETER	Optimizer parameter changes in the last 7 days	5
DBA_TAB_STATISTICS, DBA_IND_STATISTICS, DBA_TAB_COL_STATISTICS	Statistics status (stale or not), last analyzed	5, 6
DBA_TABLES, DBA_INDEXES, DBA_IND_COLUMNS, DBA_IND_PARTITIONS, DBA_IND_SUBPARTITIONS	Table and index properties, index validity	5, 6
DBA_SEGMENTS, DBA_OBJECTS	Object sizes, fragmentation estimate, created and last DDL time	5
DBA_TAB_MODIFICATIONS	Inserts, updates, deletes since last statistics	5, 6
DBA_SQL_PLAN_BASELINES	Whether SQL plan baselines exist	5, 6
ALL_USERS, ALL_TAB_COLUMNS, ALL_IND_COLUMNS	Fallbacks when DBA_* views are not granted	5, 6

Confirmation: no LLM is used to generate the report

I confirm that the report is not produced by any LLM or other AI service. It is produced entirely by the SQL*Plus / PL/SQL script, which queries the Oracle views listed above and writes the HTML file directly. The findings and recommendations come from fixed rules and thresholds written in the script: a finding appears only when its measured evidence crosses a stated threshold, and every figure in the report is a value read from the database. A review of the script source found no calls to an AI or LLM service and no outbound network calls from the script itself. The only operating-system call is the optional e-mail helper (sqlid_mail.py), controlled by the send_mail switch.